

Conseal — Problem 3 Execution Handbook

How to use this handbook

This is a build-in-order playbook, not a spec dump. You build ONE layer, you TEST it against a fixed gate, and only then you move to the next. Each milestone has a copy-paste prompt for Claude Code and a stop-and-test gate you must pass before continuing. Do not let Claude build the whole pipeline at once — the entire point of the layered design is that each layer is independently verifiable, and that property is worthless if you wire them all up blind.

Golden rule: if a test gate fails, fix it before the next milestone. Never stack an unverified layer on an unverified layer.

The detection pipeline is infrastructure. The thing being judged is the correction experience on top of it. Spend accordingly: get the pipeline correct but lean, and protect the UX hours fiercely.

The shape of what you're building

A document flows through layers ordered cheap-and-certain first, expensive-and-contextual last. Each layer only has to reason about what earlier layers left ambiguous. A final merge step differentiates and ranks every candidate. Then the user chooses: redact (content is structurally gone) or anonymize (content is replaced with a consistent placeholder). The correction UI lets a fast, over-trusting reviewer fix the tool's misses without slowing down.

```
ingest -> [OCR if raster] -> regex -> dictionary -> NER -> LLM arbitration
      -> MERGE/DIFFERENTIATE -> {REDACT | ANONYMIZE} -> correction UX
```

The redact/anonymize toggle is a first-class user choice, not a config flag. Redaction removes the characters entirely (build output from the gaps between spans). Anonymization replaces them with stable, type-aware tokens ([PERSON_1] stays [PERSON_1] everywhere) so the document stays readable and internally consistent. Same detected spans feed both; only the output stage differs.

Before any milestone: set up the project

Create the repo and the contract first. Everything downstream depends on one shared data shape, defined once and mirrored on both sides.

PROMPT — PASTE INTO CLAUDE CODE

We are building "Conseal", a layered PII detection and redaction/anonymization tool. I will build it ONE layer at a time and test each before moving on. Do not implement detection layers yet.

For this first step only:

1. Scaffold a repo: Python + FastAPI backend, React + Vite + TypeScript + Tailwind frontend, runnable with two commands. Add a README with exact run instructions.
2. Define the SHARED CONTRACT once, mirrored in backend (Pydantic) and frontend (TypeScript):
 - PiiType enum: PERSON, ORG, EMAIL, PHONE, ADDRESS, ID_NUMBER, OTHER. (Keep it small. Do not add more types speculatively.)
 - Span { id, start, end, text, type, confidence (0-1), source ("regex"|"dictionary"|"ner"|"llm"|"manual"), reason }
 - OutputMode enum: REDACT, ANONYMIZE
 - SessionState { session_id, original_text, spans[], output_mode }
3. Define the offset rule and write it in a comment: offsets are codepoint indices into original_text; assert text == original_text[start:end] everywhere a span is created.
4. Architecture rule, enforce it: thin routes, fat services. Detection layers live in separate service modules. Typed error responses, never a bare 500.

Do not build any detection logic. Stop after the scaffold + contract compile and the two-command run works.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- Both servers start with the documented commands from a clean clone.
- The contract types compile on both sides.
- There is genuinely NO detection logic yet. If Claude added a regex "to be helpful", make it delete it. The discipline starts now.

Agents to create

You do not need many. Over-staffing is the same mistake as an over-broad type system. Create these as Claude Code subagents (custom agents), each with a tight remit. Most work is sequential; the contract is what keeps parallel work from drifting, so freeze it before anyone parallelizes.

► Agent 1 — Contract Keeper

- Responsibility: owns the shared contract files (Pydantic models + TS types) and the offset rule. Nothing else may edit these.
- Inputs: requests to change a shape.
- Outputs: an updated, mirrored contract + a note on what downstream breaks.
- Constraint: never adds a type or field speculatively; only what a milestone actually needs.
- Success: backend and frontend never disagree about a shape.

► Agent 2 — Pipeline Engineer

- Responsibility: implements one detection layer at a time (regex, then dictionary, then NER, then LLM, then merge).
- Inputs: the frozen contract + the current milestone's prompt + the test fixtures.
- Outputs: one service module + its unit test, passing.
- Constraint: may NOT start a layer until the previous layer's test gate is green. May not touch the contract (asks Contract Keeper).
- Success: each layer's test passes in isolation; the merge step's precedence is explicit and tested.

► Agent 3 — Output Engineer

- Responsibility: the redact vs anonymize engines and the toggle.
- Inputs: contract + a final span list.
- Outputs: redaction engine (gap-based removal), anonymization engine (stable token substitution), one switch driven by OutputMode.
- Constraint: redaction must be proven by absence (redacted text never appears in output), not by visual cover. Anonymization tokens must be stable per entity.
- Success: a test asserts redacted spans are absent from output, and that the same entity maps to the same token everywhere under anonymize.

► Agent 4 — OCR Engineer (raster path, time-boxed, optional)

- Responsibility: detect raster pages, OCR them with bounding boxes, pixel-level redaction.
- Inputs: a scanned sample PDF.
- Outputs: text + per-token boxes; a re-rendered redacted image where content is genuinely unrecoverable.
- Constraint: HARD time box. If pixel redaction isn't provably clean by the cap, cut it and document the omission.
- Success: opening the output reveals no recoverable text/data under the boxes.

► Agent 5 — UX Engineer

- Responsibility: the correction experience — risk-ranked queue, document view, one-key fixes, duplicate-occurrence linking, the redact/anonymize toggle in the UI, and the export gate.
- Inputs: contract + a live pipeline returning real spans.
- Outputs: a wired frontend where every named state is reachable and looks intentional.
- Constraint: built against the real API, not a guessed shape. Gets the most time.
- Success: a fast reviewer can fix every miss in a few keystrokes; export is gated when high-risk items are unresolved.

► Agent 6 — Reviewer / Demo (last)

- Responsibility: fresh-eyes review against known failure modes; then the demo script + half-page writeup.
- Inputs: the finished build.
- Outputs: a short fix list (not a rewrite) + a writeup leading with the user's psychology and the redaction-vs-anonymization distinction, with the NER benchmark table as evidence.
- Constraint: does not touch code while writing the demo/writeup.
- Success: no path to export bypasses the gate; writeup reads judgment-first.

Parallelization: keep it minimal. Freeze the contract, build the pipeline sequentially (each gated), then UX builds against the live API. Backend and frontend in parallel against a still-moving contract is false efficiency on a short clock.

Skills to create

Skills are reusable instruction files Claude reads on demand. Create these so you don't re-explain the rules every milestone.

► Skill: pii-contract

- When to use: any time a span, type, or offset is involved.
- Contents: the exact contract shapes, the codepoint-offset rule, the assertion requirement, and the precedence table the merge step uses.

► Skill: layer-discipline

- When to use: starting any detection layer.
- Contents: the cheap-to-expensive ordering, the rule that each layer only handles what earlier layers left, the requirement that every layer emits a per-candidate reason and confidence, and the "do not build the next layer until this one's gate is green" rule.

► Skill: redaction-vs-anonymization

- When to use: the output stage.
- Contents: the precise distinction (remove vs replace), the gap-based removal mechanism for redaction, the stable-token rule for anonymization, and the raster pixel-redaction warning (never an overlay on recoverable data).

► Skill: test-gates

- When to use: end of every milestone.
- Contents: the gate format, and the rule that a failing gate blocks the next milestone.

The fixtures (build these once, early)

Before the detection layers, create a small test corpus. Hand-label it. This is your oracle and your benchmark set in one.

PROMPT — PASTE INTO CLAUDE CODE

Create a fixtures folder with:

1. sample-document.txt — a realistic case-file paragraph containing: a person's name appearing TWICE (once with a title, once bare), a phone number appearing TWICE in different formats, an email, a street address, one organization name that is NOT sensitive (should be allowlisted), and one person referred to only relationally ("her supervisor, [Name]") with no formatting cue.
2. labels.json — the ground-truth PII spans (text, type, char start/end), hand-verified so text == sample[start:end].
3. A couple of adversarial cases noted in labels.json: a name with no anchor, and the duplicate phone in two formats.

Verify every labeled offset programmatically before finishing. Print any mismatch.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- A script prints "all offsets valid" – every labeled span's text matches the slice.
- The duplicate name and duplicate phone are both labeled at BOTH occurrences.
- The non-sensitive org is present and labeled as allowlisted/not-PII.

Milestone 1 — Regex layer

PROMPT — PASTE INTO CLAUDE CODE

Use the layer-discipline and pii-contract skills.

Implement ONLY the regex detection layer as a separate service module. It detects structured PII over the document text: PHONE, EMAIL, ID_NUMBER (SSN/account-shaped, Luhn-validate account numbers), and similar high-precision shapes. For each hit emit a Span with source="regex", confidence near 1.0, and a reason naming the pattern.

Also: when a phone/email is found, normalize it (digits-only for phones, lowercase for emails) and record that normalized value – we will use it later to link duplicate occurrences. Do not implement linking yet, just record the normalized value.

Write a unit test that runs the regex layer over fixtures/sample-document.txt and asserts it finds every structured-PII span in labels.json, with correct offsets, and finds NO names or org names (those are not regex's job). Do not build any other layer.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- Regex test green: both phone occurrences found (both formats), email found, any ID found.
- It does NOT flag the person name or the org (precision check – regex must not overreach).
- Every emitted span asserts text == original[start:end].

Milestone 2 — Dictionary / gazetteer layer

PROMPT — PASTE INTO CLAUDE CODE

Use the layer-discipline skill.

Implement ONLY the dictionary layer as a separate service. Two jobs:

1. Match closed-world entities from a provided list (e.g. known case IDs, known names) -> Spans, source="dictionary".
2. ALLOWLIST: maintain a set of PII-shaped-but-not-sensitive strings (the non-sensitive org from fixtures, court names, etc). Produce a list of allowlisted ranges that LATER layers must suppress.

Write a unit test: over the fixture, the dictionary layer flags any known-list entity and correctly marks the non-sensitive org as allowlisted. Do not modify regex. Do not build NER yet.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- Known-list entities flagged; allowlist correctly covers the non-sensitive org.
- Running regex + dictionary together produces no duplicate/overlapping spans for the same text.

Milestone 3 — NER layer (benchmark, then ship one)

This is where your "benchmark NER" instinct pays off. Benchmark briefly, pick one, KEEP the table for the writeup.

PROMPT — PASTE INTO CLAUDE CODE

Use the layer-discipline skill.

Step 1 (benchmark, throwaway script): run TWO candidate NER approaches over the fixture labels — GLiNER (zero-shot, pass our PiiType labels at inference) and Presidio's built-in recognizer. For each, compute precision / recall / F1 and rough latency against labels.json (PERSON, ORG, ADDRESS). Print a small comparison table and save it to fixtures/ner-benchmark.md. Include the adversarial cases.

Step 2: pick the winner (favor recall on PERSON/ADDRESS and acceptable latency) and implement ONLY that one as the NER service, source="ner", with calibrated confidences and a reason. Respect the dictionary allowlist: do NOT emit spans inside allowlisted ranges.

Write a unit test: NER finds the names (both occurrences) and the address, and does NOT flag the allowlisted org. Do not build the LLM layer yet.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- ner-benchmark.md exists with a real P/R/F1 + latency table for both candidates.
- The shipped NER finds both name occurrences and the address.
- It suppresses the allowlisted org (allowlist is respected across layers).

Milestone 4 — LLM arbitration layer (sliding window)

PROMPT — PASTE INTO CLAUDE CODE

Use the layer-discipline skill.

Implement ONLY the LLM layer as a separate service, used as an ARBITRATOR, not a primary detector. It runs in two narrow cases:

1. Catch relational/implicit PII that structure-blind layers miss (e.g. "her supervisor, <Name>") — only on passages where earlier layers found nothing but a trigger word (name/medical/financial context) is present.
2. Resolve disagreement: where dictionary says allowlisted but NER says PERSON on the same span, ask the LLM with the surrounding sentence.

Use a sliding window over long text with overlap STRICTLY LARGER than the longest expected entity, so no entity is truncated at a chunk boundary; dedupe overlapping hits by offset. Make the model a flag: MODEL=cloud (my API key) or MODEL=gemma-local (Gemma 3n E4B via Ollama) — same code path. Validate the model's JSON output and re-anchor every returned span to real offsets (assert the slice matches). Emit source="llm", lower default confidence (no structural backup).

Write a test using a deterministic mock LLM that returns the relational name, so it passes offline. Do not build the merge step yet.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- With the mock LLM, the relational name ("her supervisor, <Name>") is recovered with valid offsets.
- Window overlap is larger than the longest entity (verify a name straddling a boundary is still caught).
- Switching MODEL between cloud and gemma-local does not change the code path, only the backend.

Milestone 5 — Merge / differentiation

The step you named explicitly. One function, all candidates in, one ranked list out.

PROMPT — PASTE INTO CLAUDE CODE

Use the pii-contract skill (precedence table).

Implement the merge/differentiation step as ONE pure, tested function: input = all candidate spans from regex + dictionary + NER + LLM; output = one final ranked span list.

Precedence:

- regex + dictionary = near-certain, win ties.
- NER vs LLM disagree on a span -> LLM decision breaks it (it had the sentence context).
- LLM-only catch with no structural backup -> keep but lower severity.
- Coalesce overlapping spans (union the sources, keep highest confidence, merge reasons).
- Link duplicates: same normalized value at multiple visible spots -> tie them together (record the linked occurrence offsets) so the UI can fix all at once.

Output ranking = risk tiers: (1) high-risk / linked-duplicate / structural agreement on sensitive PII, (2) low-confidence, (3) LLM-only soft review.

Write a test over the fixture asserting: both phone occurrences are linked, both name occurrences are present and the bare one isn't dropped, the relational name is tier-3 soft, and the allowlisted org appears nowhere in the final list.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- Final list: duplicates linked, no allowlisted org, relational name present but soft.
- The risk tiers map cleanly – these SAME tiers will drive the UI queue. Confirm the vocabulary is shared, not reinvented in the frontend.

Milestone 6 — Redact / anonymize toggle (the output stage)

PROMPT — PASTE INTO CLAUDE CODE

Use the redaction-vs-anonymization skill.

Implement the output stage driven by OutputMode, exposed as a user toggle (not a config):

- REDACT: build the output text ONLY from the gaps between accepted spans. Characters inside [start,end) are never copied. Prove removal with a test (redacted text absent from output; non-PII context like "Policy number:" preserved).
- ANONYMIZE: replace each accepted span with a stable, type-aware token. The SAME entity maps to the SAME token everywhere ([PERSON_1] consistently), so the doc stays readable and internally consistent. Different entities of the same type get distinct numbers.

Expose POST /api/export { session_id, accepted_span_ids[], output_mode } -> { output_text, legend[] }. Export is computed server-side from the submitted span set so "what's shared" is authoritative.

Write tests for BOTH modes: redact = target strings absent; anonymize = target strings replaced by stable tokens, same entity -> same token at all occurrences.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- REDACT: every accepted span's text is absent from output_text; surrounding context intact.
- ANONYMIZE: same person -> same [PERSON_n] at both occurrences; same phone -> same token both times.
- Flipping the toggle re-renders correctly without re-detecting.

Milestone 7 — OCR / raster path (time-boxed, optional)

PROMPT — PASTE INTO CLAUDE CODE

Use the redaction-vs-anonymization skill (raster warning).

HARD TIME BOX. Implement the raster path:

1. Ingestion: classify each page as digital-text vs raster (near-zero extractable text -> raster). Add a --force-ocr override.
2. OCR raster pages with Tesseract (pytesseract) to get text AND per-token bounding boxes.
3. Feed OCR'd text through the SAME pipeline (milestones 1-5).
4. Raster REDACTION: re-render the page image with the box regions overwritten in pixels, then re-encode. NEVER an overlay on top of recoverable image/text data.

Prove it: open the output and confirm there is no recoverable text or data under the boxes. If you cannot prove clean pixel redaction within the time box, STOP, leave the digital path as the live demo, and write a one-line note that raster redaction is a documented deferral.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- A scanned sample is classified raster, OCR'd, and run through the pipeline.
- The redacted output image has NO recoverable content under the boxes (verify by extraction attempt).
- If unproven by the time cap: cut it, keep the digital path live, document the omission. This is a tradeoff point, not a failure.

Milestone 8 — Correction UX (the actual deliverable — most time)

PROMPT — PASTE INTO CLAUDE CODE

Use the pii-contract skill. Build against the LIVE API, not a guessed shape.

Build the correction experience for a fast, over-trusting reviewer:

- Landing view = the risk-ranked queue (the tiers from the merge step), highest risk first. Each item: sentence context + one-line reason + one-key action.
- Duplicate-occurrence linking visible: fixing a linked item shows "fixed in N places" and resolves all at once.
- Document view secondary: shows redactions/anonymizations + flagged misses; confidence as opacity.
- One-key reversible gestures (accept, flip, mark-missed-by-selecting-text); a flat action log so a slip never silently destroys a correction.
- The REDACT/ANONYMIZE toggle lives here, visibly, and re-renders the preview live.
- EXPORT GATE: when high-risk items are unresolved, the export control changes state inline ("N unresolved – review first") and needs a second deliberate action; list the unresolved items in place. No blocking modal.
- Honest empty/degraded states: report which layers ran; never declare the doc "clean".

Make every named state reachable and intentional.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- Queue is risk-first; the most dangerous miss is item #1, not buried.
- Fixing a linked duplicate resolves all occurrences at once.
- Toggling redact/anonymize updates the preview without re-detecting.
- Export is gated when risky items are open; the gate is inline, not a modal.

Milestone 9 — Review, demo, writeup

PROMPT — PASTE INTO CLAUDE CODE

Act as a fresh-eyes reviewer, then as the demo/writeup author. Do not rewrite.

Review pass: find any path to export that bypasses the gate; any layer that silently no-ops without surfacing it; any dead scope from earlier milestones; any place the redact/anonymize toggle produces inconsistent tokens. Output a SHORT fix list.

Then write: (a) a demo script whose peak is the linked-duplicate fix (one key closes two leaks) and the export gate; (b) a half-page writeup that LEADS with the user's psychology (over-trust, the invisible miss), THEN the redaction-vs-anonymization distinction as a deliberate product decision, THEN the NER benchmark table as evidence of judgment, THEN the cut list (what you didn't build and why – raster path if cut, fuzzy name matching, etc). Architecture last.

STOP & TEST — MUST PASS BEFORE NEXT MILESTONE

STOP & TEST

- No export path bypasses the gate.
- The writeup's first paragraph is about the human, not the architecture.
- The benchmark table and the cut list are both present.

What to deliberately NOT build

Name these in the writeup; they're judgment signals, not gaps.

- Fuzzy / nickname name matching (honest precision limit).
- Generic date/year flagging (too noisy; trains the user to ignore flags).
- User-tunable confidence thresholds (invites cry-wolf).
- Full undo timeline (a flat action log is enough; a navigable timeline adds click-fear).
- Auth, persistence, theming, mobile (zero rubric value here).
- Multi-document batch (that's Maya's problem, not Sam's).

The one-paragraph reminder

The pipeline is the part you were told not to over-invest in. It exists to feed trustworthy candidates into the correction experience, which is the part actually being judged. Build the layers cleanly, gate every one, keep the NER benchmark as cheap evidence of judgment, make the redact/anonymize distinction a real user choice — and then put your remaining hours into the queue, the linked-duplicate moment, and the export gate. That's what a judge remembers five demos later.